Introducción a las Redes Neuronales

Alvaro J. Riascos Villegas

Uniandes y Quantil

Octubre de 2026

Contenido

1 Introducción a las redes neuronales

2 Ideas principales

IA en Contexto

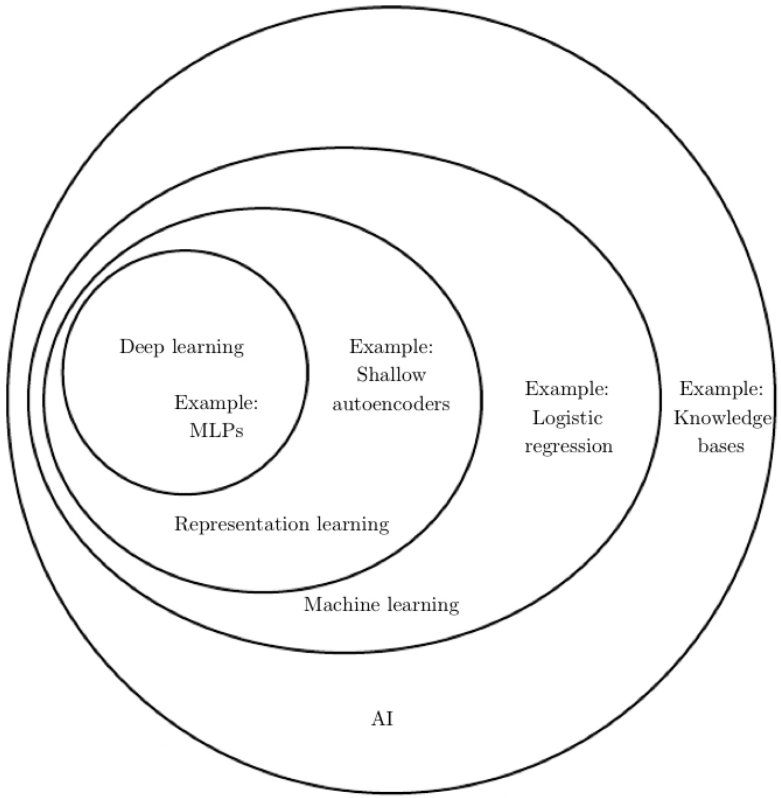


Figure 1.4: A Venn diagram showing how deep learning is a kind of representation learning, which is in turn a kind of machine learning, which is used for many but not all approaches to AI. Each section of the Venn diagram includes an example of an AI technology.

Fuente: Goodfellow et al. (2016), fig. 1.4.

Historia IA

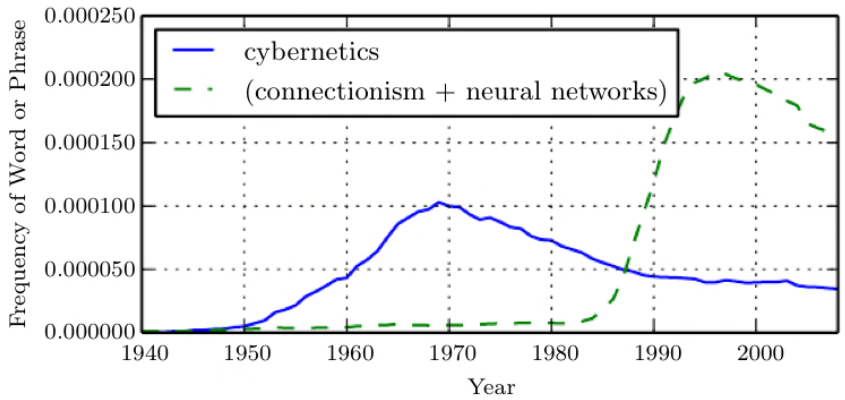


Figure 1.7: The figure shows two of the three historical waves of artificial neural nets research, as measured by the frequency of the phrases "cybernetics" and "connectionism" or "neural networks" according to Google Books (the third wave is too recent to appear). The first wave started with cybernetics in the 1940s–1960s, with the development of theories of biological learning (McCulloch and Pitts, 1943; Hebb, 1949) and implementations of the first models such as the perceptron (Rosenblatt, 1958) allowing the training of a single neuron. The second wave started with the connectionist approach of the 1980–1995 period, with back-propagation (Rumelhart *et al.*, 1986a) to train a neural network with one or two hidden layers. The current and third wave, deep learning, started around 2006 (Hinton *et al.*, 2006; Bengio *et al.*, 2007; Ranzato *et al.*, 2007a), and is just now appearing in book form as of 2016. The other two waves similarly appeared in book form much later than the corresponding scientific activity occurred.

Fuente: Goodfellow et al. (2016), fig. 1.7.

Disponibilidad de Datos

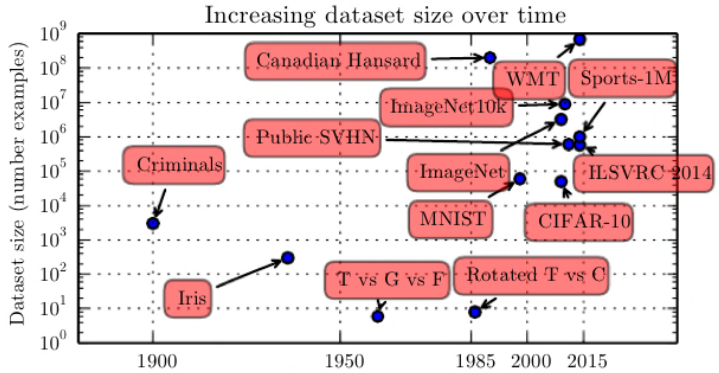


Figure 1.8: Dataset sizes have increased greatly over time. In the early 1900s, statisticians studied datasets using hundreds or thousands of manually compiled measurements (Garson, 1900; Gosset, 1908; Anderson, 1935; Fisher, 1936). In the 1950s through 1980s, the pioneers of biologically inspired machine learning often worked with small, synthetic datasets, such as low-resolution bitmaps of letters, that were designed to incur low computational cost and demonstrate that neural networks were able to learn specific kinds of functions (Widrow and Hoff, 1960; Rumelhart *et al.*, 1986b). In the 1980s and 1990s, machine learning became more statistical in nature and began to leverage larger datasets containing tens of thousands of examples such as the MNIST dataset (shown in Fig. 1.9) of scans of handwritten numbers (LeCun *et al.*, 1998b). In the first decade of the 2000s, more sophisticated datasets of this same size, such as the CIFAR-10 dataset (Krizhevsky and Hinton, 2009) continued to be produced. Toward the end of that decade and throughout the first half of the 2010s, significantly larger datasets, containing hundreds of thousands to tens of millions of examples, completely changed what was possible with deep learning. These datasets included the public Street View House Numbers dataset (Netzer *et al.*, 2011), various versions of the ImageNet dataset (Deng *et al.*, 2009, 2010a; Russakovsky *et al.*, 2014a), and the Sports-1M dataset (Karpathy *et al.*, 2014). At the top of the graph, we see that datasets of translated sentences, such as IBM's dataset constructed from the Canadian Hansard (Brown *et al.*, 1990) and the WMT 2014 English to French dataset (Schwenk, 2014) are typically far ahead of other dataset sizes.

Fuente: Goodfellow et al. (2016), fig. 1.8.

Base de datos MNIST

Fuente: Goodfellow et al. (2016), fig. 1.9; datos MNIST: LeCun et al. (1998).

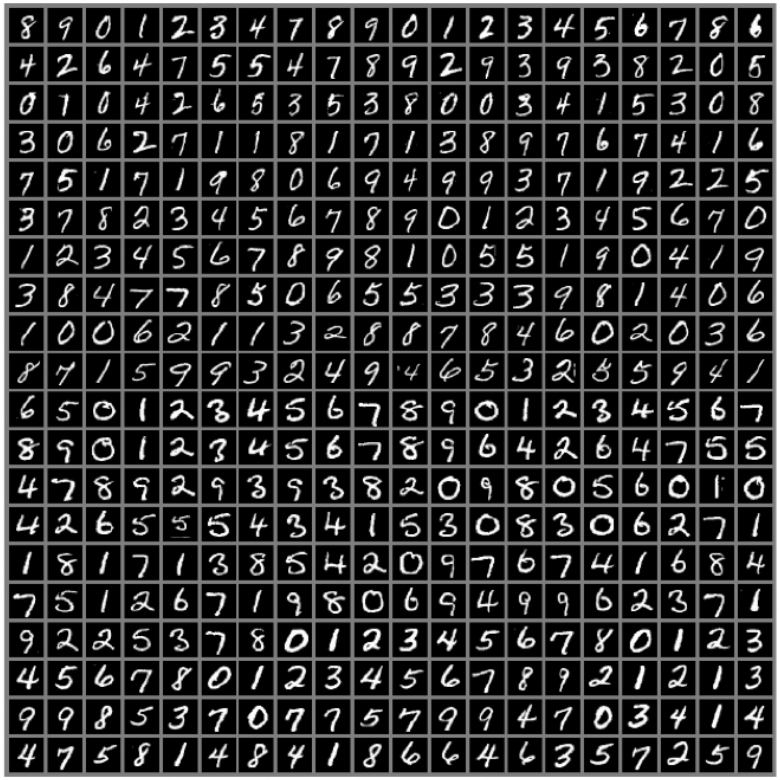


Figure 1.9: Example inputs from the MNIST dataset. The "NIST" stands for National Institute of Standards and Technology, the agency that originally collected this data. The "M" stands for "modified," since the data has been preprocessed for easier use with machine learning algorithms. The MNIST dataset consists of scans of handwritten digits and associated labels describing which digit 0-9 is contained in each image. This simple classification problem is one of the simplest and most widely used tests in deep learning research. It remains popular despite being quite easy for modern techniques to solve. Geoffrey Hinton has described it as "the *drosophila* of machine learning," meaning that it allows machine learning researchers to study their algorithms in controlled laboratory conditions, much as biologists often study fruit flies.

Conexiones

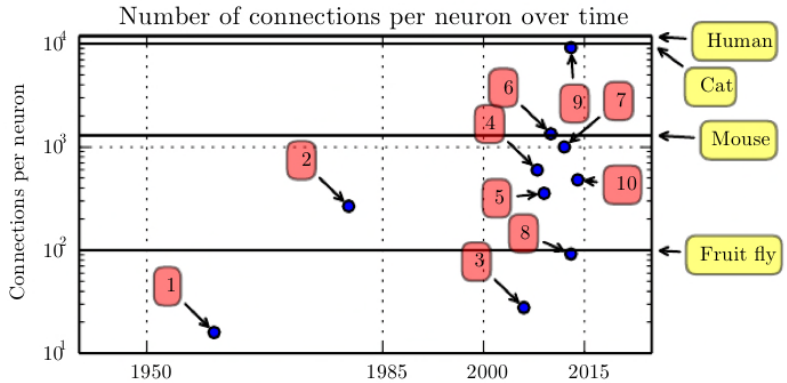


Figure 1.10: Initially, the number of connections between neurons in artificial neural networks was limited by hardware capabilities. Today, the number of connections between neurons is mostly a design consideration. Some artificial neural networks have nearly as many connections per neuron as a cat, and it is quite common for other neural networks to have as many connections per neuron as smaller mammals like mice. Even the human brain does not have an exorbitant amount of connections per neuron. Biological neural network sizes from Wikipedia (2015).

1. Adaptive linear element (Widrow and Hoff, 1960)
2. Neocognitron (Fukushima, 1980)
3. GPU-accelerated convolutional network (Chellapilla *et al.*, 2006)
4. Deep Boltzmann machine (Salakhutdinov and Hinton, 2009a)
5. Unsupervised convolutional network (Jarrett *et al.*, 2009)
6. GPU-accelerated multilayer perceptron (Ciresan *et al.*, 2010)
7. Distributed autoencoder (Le *et al.*, 2012)
8. Multi-GPU convolutional network (Krizhevsky *et al.*, 2012)
9. COTS HPC unsupervised convolutional network (Coates *et al.*, 2013)
10. GoogLeNet (Szegedy *et al.*, 2014a)

Fuente: Goodfellow et al. (2016), fig. 1.10.

Tamaño Redes

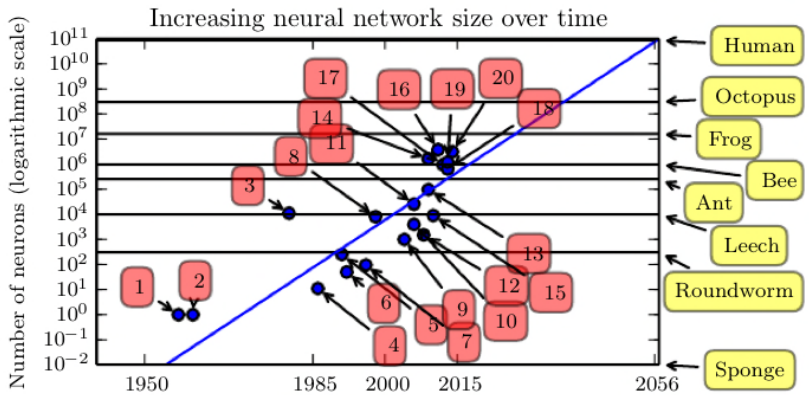


Figure 1.11: Since the introduction of hidden units, artificial neural networks have doubled in size roughly every 2.4 years. Biological neural network sizes from Wikipedia (2015).

1. Perceptron (Rosenblatt, 1958, 1962)
2. Adaptive linear element (Widrow and Hoff, 1960)
3. Neocognitron (Fukushima, 1980)
4. Early back-propagation network (Rumelhart *et al.*, 1986b)
5. Recurrent neural network for speech recognition (Robinson and Fallside, 1991)
6. Multilayer perceptron for speech recognition (Bengio *et al.*, 1991)
7. Mean field sigmoid belief network (Saul *et al.*, 1996)
8. LeNet-5 (LeCun *et al.*, 1998b)
9. Echo state network (Jaeger and Haas, 2004)
10. Deep belief network (Hinton *et al.*, 2006)
11. GPU-accelerated convolutional network (Chellapilla *et al.*, 2006)
12. Deep Boltzmann machine (Salakhutdinov and Hinton, 2009a)
13. GPU-accelerated deep belief network (Raina *et al.*, 2009)
14. Unsupervised convolutional network (Jarrett *et al.*, 2009)
15. GPU-accelerated multilayer perceptron (Ciresan *et al.*, 2010)
16. OMP-1 network (Coates and Ng, 2011)
17. Distributed autoencoder (Le *et al.*, 2012)
18. Multi-GPU convolutional network (Krizhevsky *et al.*, 2012)
19. COTS HPC unsupervised convolutional network (Coates *et al.*, 2013)
20. GoogLeNet (Szegedy *et al.*, 2014a)

Fuente: Goodfellow et al. (2016), fig. 1.11.

Arquitectura Sistema de IA

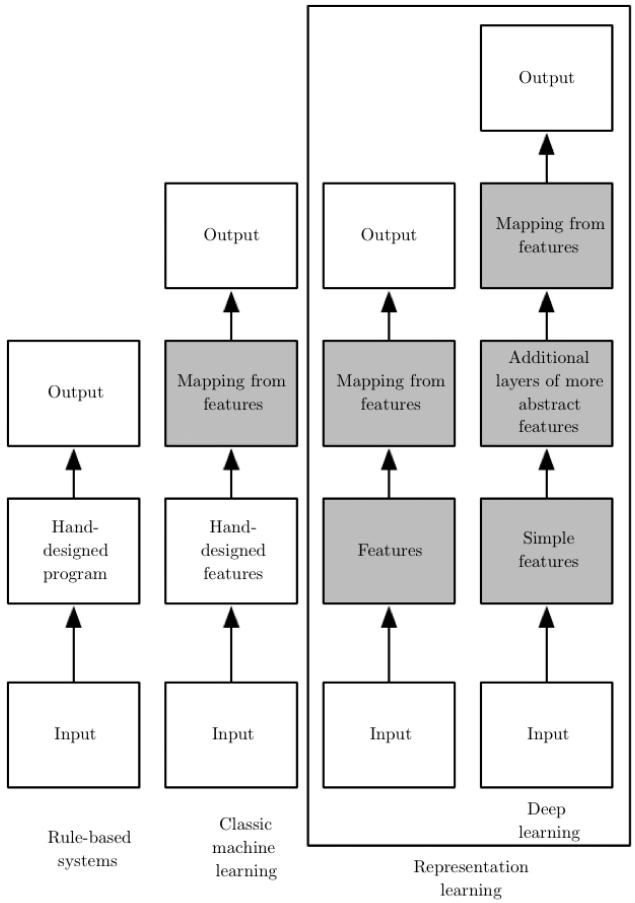


Figure 1.5: Flowcharts showing how the different parts of an AI system relate to each other within different AI disciplines. Shaded boxes indicate components that are able to learn from data.

Fuente: Goodfellow et al. (2016), fig. 1.5.

Representaciones

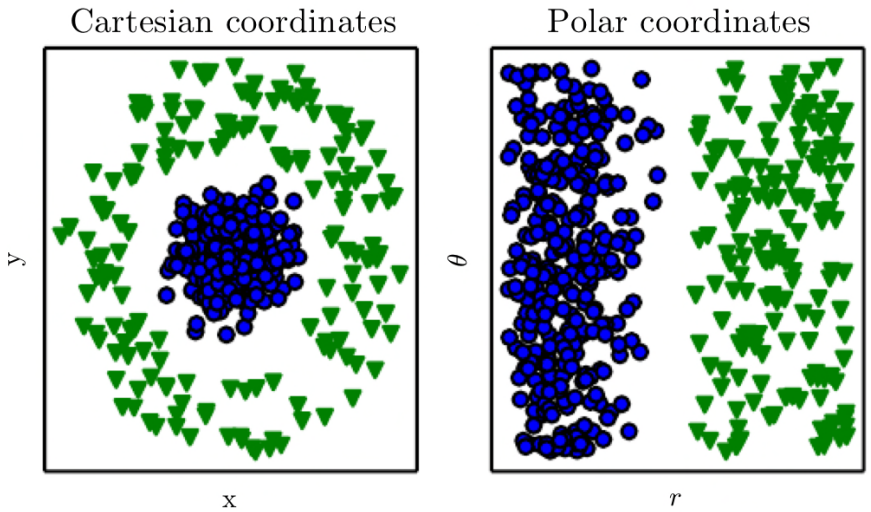


Figure 1.1: Example of different representations: suppose we want to separate two categories of data by drawing a line between them in a scatterplot. In the plot on the left, we represent some data using Cartesian coordinates, and the task is impossible. In the plot on the right, we represent the data with polar coordinates and the task becomes simple to solve with a vertical line. (Figure produced in collaboration with David Warde-Farley)

Fuente: Goodfellow et al. (2016), fig. 1.1.

Contenido

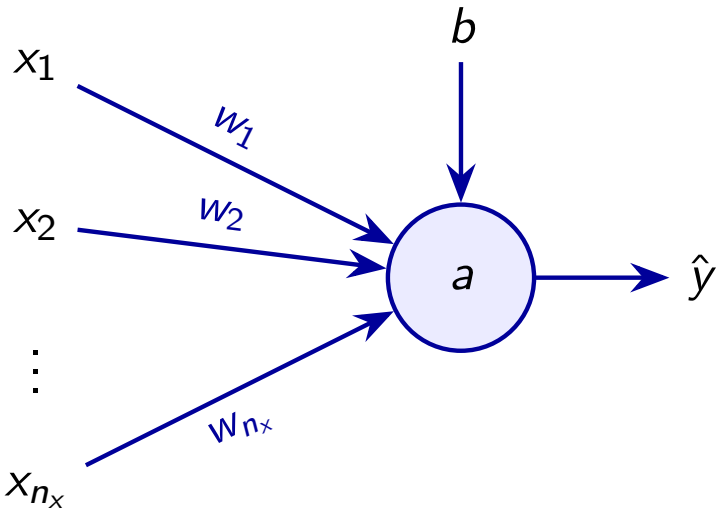
1 Introducción a las redes neuronales

2 Ideas principales

Hoja de ruta

- Redes neuronales (estructura recursiva).
- Funciones de activación.
- Función de pérdida.
- Optimización: descenso de gradiente; el gradiente se calcula con retropropagación (*backpropagation*).

La red más básica: regresión logística



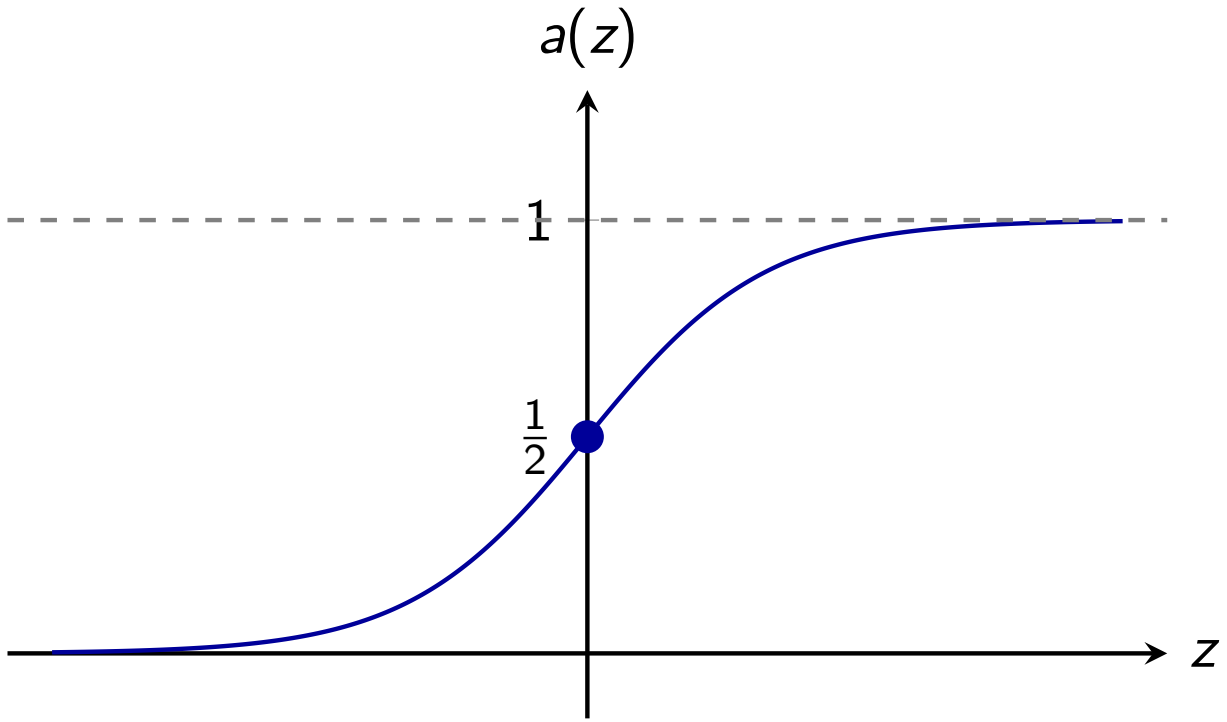
- Una sola capa (la capa de salida).
- No hay capas ocultas.
- Parámetros: pesos $w \in \mathbb{R}^{n_x}$ y sesgo b.

$$z = w^\top x + b,$$

$$\hat{y} = a(z) = \frac{1}{1 + e^{-(w^\top x + b)}}.$$

Función de activación: sigmoide

$$a(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Función de pérdida

- Datos de entrenamiento: $\tau = \{(x^{(1)}, y^{(1)}), \ldots, (x^{(m)}, y^{(m)})\}$, con $x^{(i)} \in \mathbb{R}^{n_x}$ y $y^{(i)} \in \{0, 1\}$.

Pérdida con un ejemplo

$$\mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -y^{(i)} \ln \hat{y}^{(i)} - (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}),$$

donde $\hat{y}^{(i)}$ es el pronóstico.

Función de costo

$$\begin{aligned} J(w, b) &= \frac{1}{m} \sum_{i=1}^{m} \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) \\ &= -\frac{1}{m} \sum_{i=1}^{m} \left[y^{(i)} \ln \hat{y}^{(i)} + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}) \right] \end{aligned}$$

Función de costo y máxima verosimilitud

- Supuesto: dado $x^{(i)}$, $y^{(i)} \sim \text{Bernoulli}(\hat{y}^{(i)})$, independientes entre ejemplos, con $\hat{y}^{(i)} = \sigma(w^\top x^{(i)} + b)$.

Verosimilitud y log-verosimilitud

$$V(w, b) = \prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)}; w, b) = \prod_{i=1}^{m} (\hat{y}^{(i)})^{y^{(i)}} (1 - \hat{y}^{(i)})^{1-y^{(i)}},$$

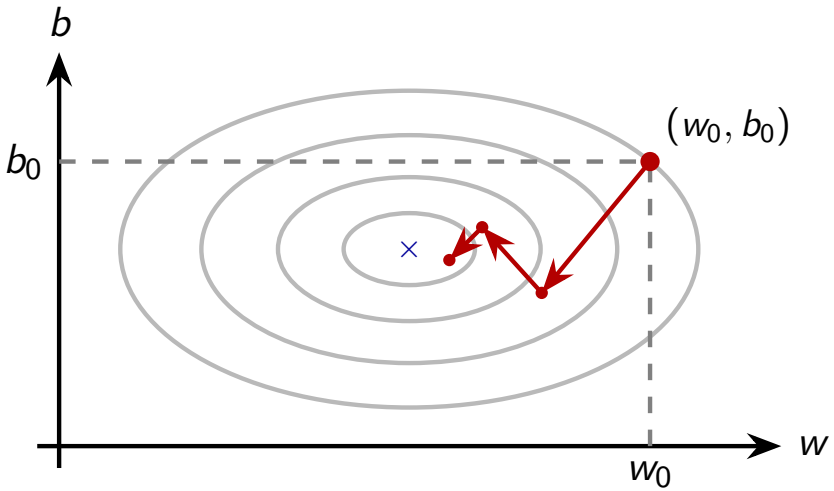
$$\ln V(w, b) = \sum_{i=1}^{m} \left[y^{(i)} \ln \hat{y}^{(i)} + (1 - y^{(i)}) \ln(1 - \hat{y}^{(i)}) \right].$$

Conclusión

$$J(w, b) = -\frac{1}{m} \ln V(w, b).$$

Minimizar la función de costo equivale a estimar (w, b) por máxima verosimilitud.

Optimización: descenso de gradiente



$$\min_{w,\, b} \; J(w, b)$$

$$w \leftarrow w - \alpha \underbrace{\frac{\partial J}{\partial w}(w_0, b_0)}_{\equiv dw}$$

$$b \leftarrow b - \alpha \underbrace{\frac{\partial J}{\partial b}(w_0, b_0)}_{\equiv db}$$

donde $\alpha > 0$ es la tasa de aprendizaje.

Cálculo del gradiente

- Obsérvese que J depende de los m ejemplos de entrenamiento. Si para calcular J y ∇J se usan todos los ejemplos, se habla de descenso de gradiente por lotes (*batch*); si se usa uno o un subconjunto, de descenso estocástico o por *mini-batch*.
- En general, el problema es calcular

$$\frac{\partial J(w,b)}{\partial w}, \qquad \frac{\partial J(w,b)}{\partial b}.$$

- Hacerlo término a término es costoso en redes grandes. La retropropagación (*backpropagation*) lo hace de forma eficiente explotando la estructura recursiva de la red y la regla de la cadena (Rumelhart et al., 1986).

Cálculo del gradiente: un ejemplo

Regresión logística con un solo ejemplo

- Clasificación binaria, $y \in \{0, 1\}$, con dos variables predictoras.

$$z = w_1 x_1 + w_2 x_2 + b, \qquad a = \sigma(z) = \hat{y},$$
$$J(w, b) = \mathcal{L}(a, y) = -y \ln a - (1 - y) \ln(1 - a).$$

- Por la regla de la cadena:

$$\frac{\partial J(w, b)}{\partial w_1} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w_1} \equiv dw_1$$

Cálculo del gradiente: derivadas con la sigmoide

Derivadas parciales

$$\frac{\partial J}{\partial a} = \frac{a-y}{a(1-a)}, \qquad \frac{\partial a}{\partial z} = a^2 e^{-z} = a(1-a),$$
$$\frac{\partial z}{\partial w_1} = x_1, \qquad \frac{\partial z}{\partial b} = 1.$$

Por lo tanto,

$$dz \equiv \frac{\partial J}{\partial z} = \frac{\partial J}{\partial a} \cdot \frac{\partial a}{\partial z} = a - y, \qquad db \equiv \frac{\partial J}{\partial b} = dz \cdot \frac{\partial z}{\partial b} = dz.$$

En resumen (activación sigmoide)

$$dz = a - y, \qquad dw_j = x_j\, dz, \qquad db = dz.$$

Cálculo del gradiente: varios ejemplos

Gradiente con m ejemplos

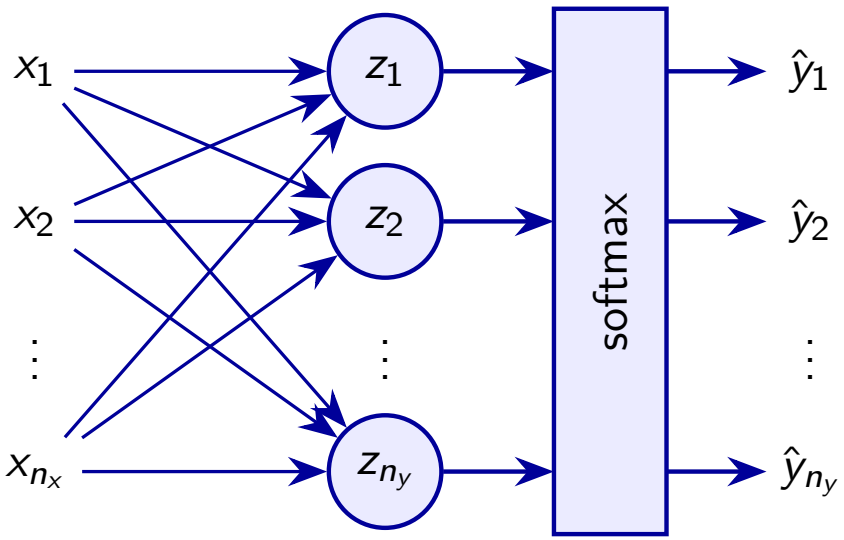
$$J(w,b) = \frac{1}{m}\sum_{i=1}^{m}\mathcal{L}(\hat{y}^{(i)}, y^{(i)}),$$

$$\frac{\partial J(w,b)}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\frac{\partial \mathcal{L}(\hat{y}^{(i)}, y^{(i)})}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\left(a^{(i)} - y^{(i)}\right)x_j^{(i)},$$

$$\frac{\partial J(w,b)}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(a^{(i)} - y^{(i)}\right),$$

donde $a^{(i)} = \sigma(z^{(i)})$.

Regresión softmax: una red de una sola capa



- Una sola capa (la capa de salida), con una neurona por clase.
- No hay capas ocultas.
- Parámetros: $W \in \mathbb{R}^{n_y \times n_x}$ (filas $w_k^\top$) y $b \in \mathbb{R}^{n_y}$.
- Con $n_y = 2$ equivale a la regresión logística.

$$z_k = w_k^\top x + b_k, \qquad k = 1, \ldots, n_y,$$

$$\hat{y}_k = \frac{e^{z_k}}{\sum_{j=1}^{n_y} e^{z_j}}, \qquad \hat{y}_k > 0, \quad \sum_{k=1}^{n_y} \hat{y}_k = 1.$$

Cálculo del gradiente: softmax

- Clasificación en n_y clases. La etiqueta se codifica como vector *one-hot*: $y \in \{0,1\}^{n_y}$, con $y_k = 1$ si el ejemplo es de la clase k.

Modelo y pérdida con un ejemplo

$$z_k = w_k^\top x + b_k, \qquad a_k = \frac{e^{z_k}}{\sum_{j=1}^{n_y} e^{z_j}} = \hat{y}_k, \qquad k = 1, \ldots, n_y,$$

$$\mathcal{L}(a, y) = -\sum_{k=1}^{n_y} y_k \ln a_k \quad \text{(entropía cruzada)}.$$

- Cada a_k depende de *todas* las z_j (por el denominador):

$$\frac{\partial a_k}{\partial z_j} = a_k(\delta_{kj} - a_j), \qquad \delta_{kj} = \begin{cases} 1 & k = j, \\ 0 & k \neq j. \end{cases}$$

- Con $n_y = 2$ se recupera la regresión logística.

Cálculo del gradiente: softmax (continuación)

Regla de la cadena

$$dz_j \equiv \frac{\partial \mathcal{L}}{\partial z_j} = \sum_{k=1}^{n_y} \frac{\partial \mathcal{L}}{\partial a_k} \frac{\partial a_k}{\partial z_j} = \sum_{k=1}^{n_y} \left(-\frac{y_k}{a_k}\right) a_k(\delta_{kj} - a_j)$$

$$= -y_j + a_j \sum_{k=1}^{n_y} y_k = a_j - y_j \qquad \left(\text{pues } \textstyle\sum_k y_k = 1\right).$$

En resumen (activación softmax)

$$dz_j = a_j - y_j, \qquad dw_{j\ell} = x_\ell \, dz_j, \qquad db_j = dz_j.$$

En forma vectorial: $dz = a - y$, $dW = dz\, x^\top$, $db = dz$. Es la misma forma que con la sigmoide: pronóstico menos observado.

Aproximación universal

- Las redes neuronales son aproximadores universales: una red con una capa oculta y capa de salida lineal puede aproximar cualquier función continua en un compacto con la precisión deseada, siempre que la función de activación sea continua y no polinómica (Cybenko, 1989; Hornik, 1991; Leshno et al., 1993).
- La precisión se logra aumentando el número de neuronas en la capa oculta.

Referencias

Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals, and Systems*, 2(4):303–314.

Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press, Cambridge, MA.

Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2):251–257.

LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324.

Leshno, M., Lin, V. Y., Pinkus, A., and Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks*, 6(6):861–867.

Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.